

FORMULA 1 RACE STRATEGY & TELEMETRY SIMULATOR PROJECT REPORT

Academic university report

Introduction

Motorsport race strategies are highly complex, data-driven optimization problems. In modern Formula 1, race outcomes are heavily dictated by split-second decisions regarding tire degradation, pit-stop windows, and varying fuel loads. With the upcoming 2026 technical regulation changes, calculating the optimal race stint has become increasingly complex. This project introduces a lightweight, object-oriented Java simulation tool designed to model these variables. By factoring in driver-specific risk profiles (aggressiveness vs. tire preservation) alongside constructor baseline performance, the simulator calculates lap-by-lap time deltas. The primary objective is to provide a mathematical forecast comparing total race times across different pit-stop strategies, enabling data-driven decision-making over a full Grand Prix distance.

Design Decisions & Rationale

The architecture was heavily influenced by the core principles of Object-Oriented Programming (OOP) to ensure modularity and scalability.

- **Inheritance & Polymorphism:** An abstract `TyreCompound` class was utilized as a blueprint, allowing `SoftTyre`, `MediumTyre`, and `HardTyre` subclasses to inherit common attributes while overriding specific degradation rates and grip advantages. This makes it trivial to add new compounds (like Wet or Intermediate tires) in the future without altering the core engine.
- **Encapsulation:** All class attributes within the `Driver` and `Constructor` models are declared as private, utilizing getter methods to prevent unauthorized or accidental modification of critical simulation parameters during runtime.
- **Data Structures:** A 2D array (`double[][]`) was selected to store the generated telemetry. This provides an optimal, lightweight structure with fast $O(1)$ time complexity for row/column lookups when passing data to the reporting module.
-

Implementation Details

The system was implemented entirely in Java, utilizing a command-line interface (CLI) to capture user configurations. The core logic resides within `SimulationEngine.java`, which runs a parameterized for loop representing the laps of the race.

During each iteration, the engine applies a mathematical degradation formula: adding a baseline pace, subtracting a fuel burn-off factor (simulating a lighter chassis), and adding a tire wear penalty that scales dynamically based on the driver's tire preservation skill. Time losses are injected at predefined array indices matching the user's chosen pit laps. Finally, the `StrategyReport` class parses the 2D array to print formatted, tabular outputs to the console.

Implementation Details

The system was implemented entirely in Java, utilizing a command-line interface (CLI) to capture user configurations. The core logic resides within `SimulationEngine.java`, which runs a parameterized for loop representing the laps of the race. During each iteration, the engine applies a mathematical degradation formula: adding a baseline pace, subtracting a fuel burn-off factor (simulating a lighter chassis), and adding a tire wear penalty that scales dynamically based on the driver's tire preservation skill. Time losses are injected at predefined array indices matching the user's chosen pit laps. Finally, the `StrategyReport` class parses the 2D array to print formatted, tabular outputs to the console.

Testing Approach

To ensure the reliability of the mathematical engine, the system was subjected to rigorous edge-case testing:

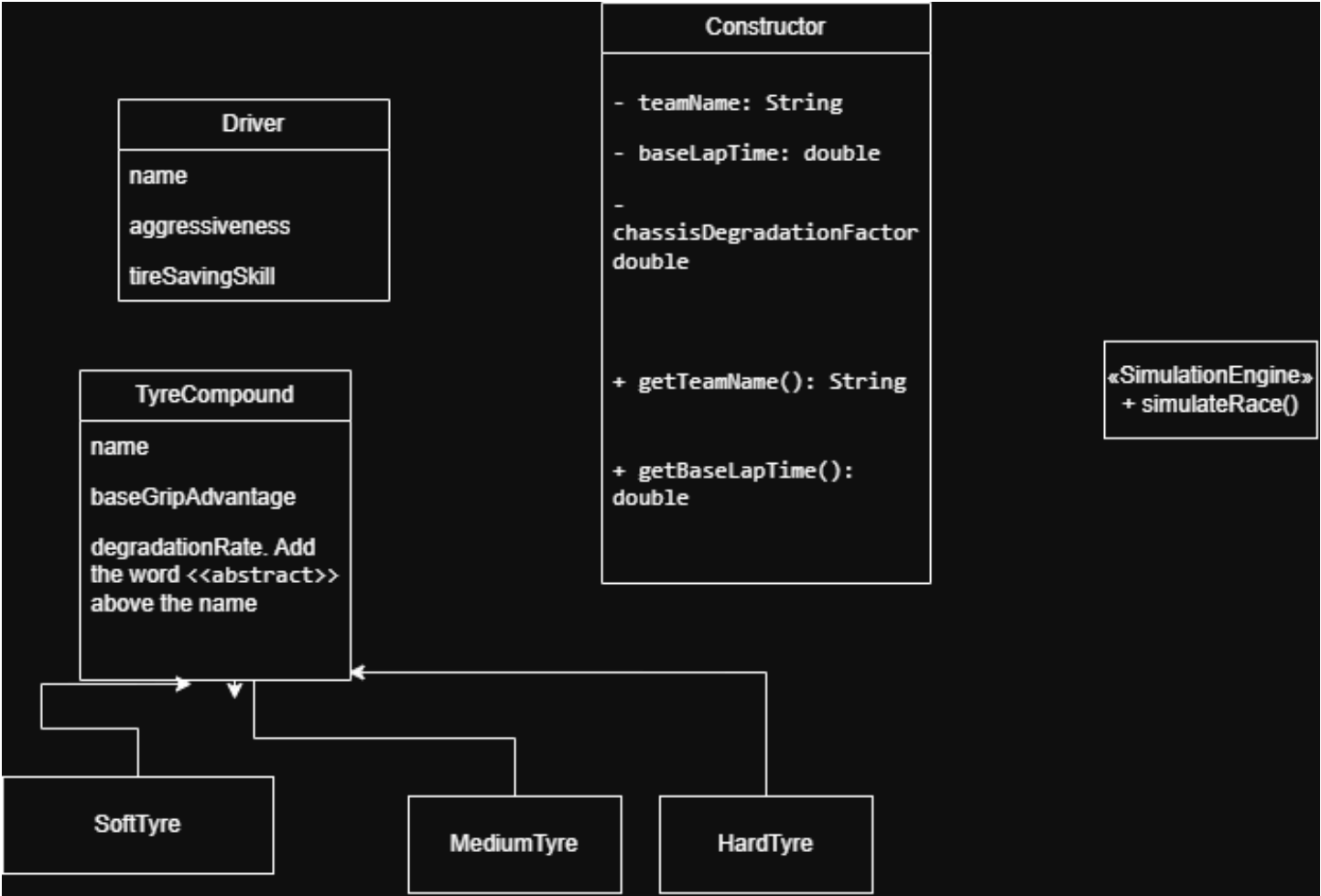
- **Input Validation:** Try-catch blocks were implemented in the Scanner input loop to catch `NumberFormatExceptions`.
- **Boundary Testing:** Driver aggressiveness and tire saving parameters were tested with extreme values (e.g., -5.0 or 10.0) to ensure the conditional logic correctly overrides out-of-bound inputs to a safe default of 1.0.
- **Logic Verification:** The pit-stop logic was tested by configuring a 0-stop strategy versus a 5-stop strategy to verify that the 22-second pit lane penalty was correctly applied strictly on the designated laps.
-

Challenges Faced

The primary challenge involved calibrating the mathematical tire degradation model. Initially, the linear wear formulas resulted in lap times that either dropped off too aggressively (causing unrealistic

20-second pace deficits) or barely degraded at all. Tuning the base pace multipliers required extensive trial and error to mimic realistic F1 telemetry. Additionally, synchronizing the stint array index with the pit-stop lap array required careful boundary checking to prevent `ArrayIndexOutOfBoundsException` exceptions during transitions between tire compounds.

•



•

Learnings & Key Takeaways

Building this simulator significantly reinforced practical applications of Java control structures and object-oriented design. Managing state transitions (like switching from a Soft to a Medium tire mid-loop) provided deep insights into array manipulation and memory reference handling. Furthermore, translating abstract real-world physics—like the relationship between fuel weight and lap time—into actionable algorithmic logic demonstrated the power of programmatic problem-solving.

Future Enhancements

To expand the realism and complexity of the simulator, several features could be introduced:

- **Machine Learning Integration:** Replacing the static degradation formulas by integrating a predictive machine learning model (e.g., utilizing Scikit-learn in Python) trained on historical CSV telemetry data to predict exact tire wear dynamically.
- **Dynamic Weather System:** Introducing a randomized weather variable that drastically alters track grip mid-race, forcing emergency pit stops for Wet tires.
- **Graphical User Interface (GUI):** Migrating the current CLI dashboard into a visual application using JavaFX, rendering lap-by-lap line graphs rather than console tables.

-

- **References**

- Oracle. (2026). *Java Documentation*.
- FIA. (2026). *Formula 1 Technical Regulations*.

-